

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA - GOIÁS

Pós-Graduação em Inteligência Artificial Aplicada - Módulo 2

Pipeline de Dados em Nuvem para Aprendizagem de Máquina

Previsão de violação de SLA em chamados de suporte técnico para apoio à decisão

Integrantes do grupo:

João Paulo A. S. Monteiro

Rogério B. dos Anjos

Pedro Felipe de Moraes Carrijo

Aprendizagem de Máquina - Prof. Dr. Sirlon Diniz

Cloud Computing - Prof. Dr. Raphael Gomes

Modelagem de Dados para IA - Prof. Dr. Otávio Calaça

Goiânia - GO, junho de 2026

Sumário

Sumário 2

1. Definição do problema e decisão apoiada 3
2. Conjuntos de dados utilizados 4
3. Arquitetura geral da solução 5
4. Processamento e extração de atributos 5
5. Pipeline de ELT (Airflow, dbt e Snowflake) 6
6. Uso de recursos da AWS 6
7. Aprendizagem de Máquina: modelos e métricas 7
8. Dashboard e principais análises 8
9. Avaliação do pipeline e análise qualitativa 9
- Análise qualitativa 10
10. Limitações e próximos passos 10
- Referências 10

1. Definição do problema e decisão apoiada

O domínio escolhido pela nossa equipe foi o de suporte técnico (service desk). Em operações de suporte, cada chamado possui um acordo de nível de serviço (SLA) de resolução, e o estouro desse prazo gera insatisfação do cliente e, muitas vezes, penalidades contratuais. O problema que atacamos é antecipar quais chamados têm maior risco de violar o SLA de resolução, de modo que o time consiga agir antes de o prazo estourar.

Os elementos do problema, conforme solicitado no item 4.1, são:

Elemento	Definição no nosso projeto
Domínio de aplicação	Suporte técnico / service desk de uma empresa de software (SaaS)

Elemento	Definição no nosso projeto
Usuário / tomador de decisão	Gestor (coordenador) da equipe de suporte
Decisão apoiada	Para onde realocar agentes e quais chamados priorizar na fila
Fontes de dados	Tickets (estruturado), descrições textuais (não estruturado) e logs (JSON)
Tarefa de ML	Classificação binária: prever sla_breach (violou ou não o SLA)
Resultado esperado	Fila priorizada por risco e indicadores para a gestão do suporte

A decisão é tomada logo após a primeira resposta ao cliente. Esse recorte é importante porque define quais atributos podemos usar sem cometer vazamento de dados: usamos apenas o que já é conhecido nesse momento e nunca o tempo de resolução em si, que é justamente o desfecho que queremos antecipar.

2. Conjuntos de dados utilizados

Atendendo ao requisito de combinar dados estruturados com pelo menos uma fonte não estruturada ou semiestruturada, trabalhamos com três fontes complementares. O esquema foi desenhado espelhando o Customer Support Ticket Dataset citado no enunciado.

Fonte	Tipo	Formato	Registros	Conteúdo principal
tickets.csv	Estruturado	CSV	8.000	categoria, prioridade, canal, tier, tempos, CSAT
ticket_messages.jsonl	Não estruturado	JSONL	8.000	texto livre do cliente + nota do agente
ticket_events.json	Semiestruturado	JSON	8.000	trilha de eventos/status de cada chamado

A variável-alvo (sla_breach) foi definida a partir do SLA de resolução por prioridade (Crítica: 4h; Alta: 8h; Média: 24h; Baixa: 48h). A prevalência de violações no conjunto ficou em torno de 22.8%, o que configura um problema desbalanceado, como é comum na prática. Documentamos origem, formato, campos e limitações dos dados no arquivo docs/dicionario_de_dados.md do repositório.

3. Arquitetura geral da solução

A solução foi desenvolvida com uma arquitetura em camadas (bronze/prata/ouro), seguindo a abordagem ELT (Extract, Load, Transform). No ambiente de desenvolvimento, usamos o DuckDB como proxy local do Snowflake (a mesma modelagem dbt roda nos dois, bastando trocar o profile), Airflow para orquestração, Python para processamento/ML e Metabase para o dashboard. O Amazon S3 é usado para o armazenamento bruto em nuvem.

Como pedido no item 4.5, também desenhamos a arquitetura equivalente 100% em serviços AWS, apresentada na figura a seguir.

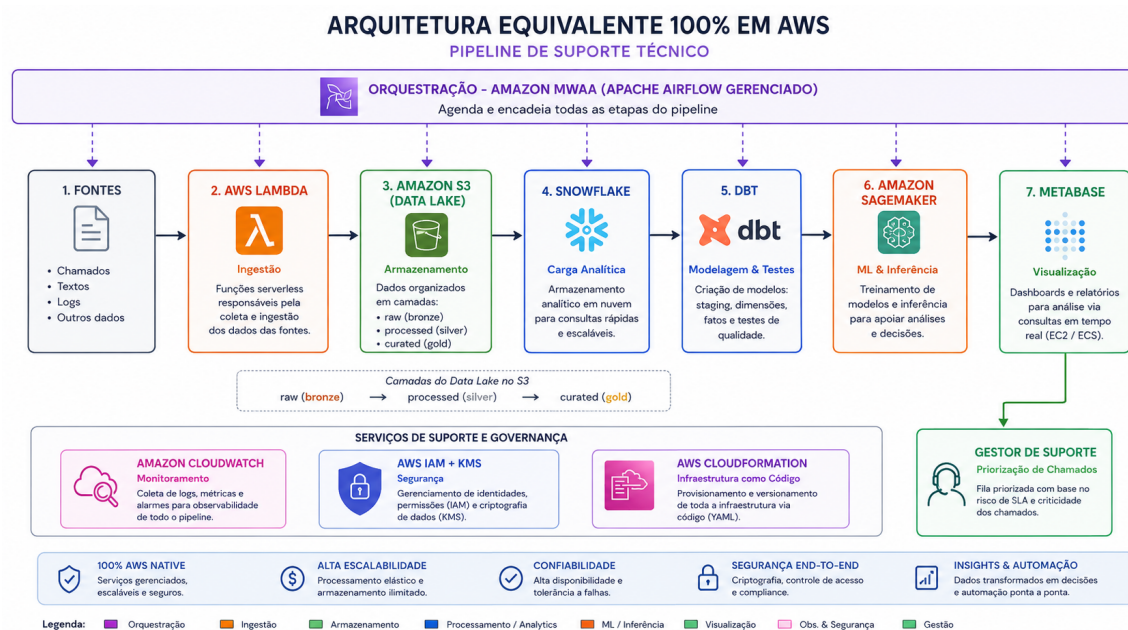


Figura 1. Arquitetura equivalente 100% em AWS.

Nessa arquitetura, a ingestão roda em AWS Lambda, os dados ficam no S3 em três camadas (raw, processed, curated), a carga e a modelagem analítica acontecem no Snowflake com dbt, o treino/inferência do modelo fica no Amazon SageMaker, a orquestração usa o Amazon MWAA (Airflow gerenciado), o dashboard roda em Metabase sobre EC2/ECS, e o monitoramento e a segurança ficam a cargo de CloudWatch e IAM/KMS. Toda a infraestrutura base é descrita como código no template CloudFormation (infra/cloudformation.yaml).

4. Processamento e extração de atributos

Para os dados estruturados, fizemos limpeza, padronização de categorias, tratamento de valores ausentes (o CSAT ausente virou uma flag, sem imputar o alvo) e derivamos partes de data (hora, dia da semana, mês) úteis para análise.

Para o texto livre (não estruturado), extraímos atributos como comprimento do texto, número de palavras, quantidade de pontos de interrogação e exclamação, proporção de letras maiúsculas, contagem de termos de um léxico de urgência e flags de palavras-chave (reembolso, cancelamento, erro). Para os logs (semiestruturado), agregamos o número de eventos, o número de mudanças de status e se houve reabertura.

O resultado é uma tabela analítica única (mart_ml_features), com 25 atributos por chamado mais o alvo, pronta para o modelo. Reforçamos que o tempo de resolução e o CSAT foram deliberadamente deixados de fora dos atributos para evitar vazamento de dados.

5. Pipeline de ELT (Airflow, dbt e Snowflake)

O pipeline é orquestrado por uma DAG do Airflow com nove tarefas, na ordem: ingestão, processamento/atributos, treino e avaliação do modelo, carga no warehouse, build do dbt (modelos mais testes), export do dashboard, publicação da camada analítica no Postgres, provisionamento do dashboard no Metabase e upload das camadas para o S3. A última execução completou as nove tarefas com sucesso (9/9), registrada em evidencias/airflow_dag_run.log. Para facilitar a execução local, também fornecemos um run_pipeline.py que roda exatamente a mesma sequência.

No dbt, organizamos os modelos em camadas:

- Staging (views): stg_tickets, stg_text_features, stg_log_features e stg_predictions;

- Dimensões: `dim_channel`, `dim_product`, `dim_category` e `dim_date`;
- Fatos e marts: `fact_tickets`, `mart_ml_features`, `fact_predictions` e `mart_ticket_decision`.

Implementamos 10 testes de qualidade de dados no dbt (acima do mínimo de 2 pedido), incluindo `not_null` e `unique` em chaves, `accepted_values` em prioridade e no alvo, e um teste de `relationships` entre as previsões e o fato de chamados. Na última execução, todos os testes passaram (PASS=22, 0 erros). A documentação dos modelos está nos arquivos `.yml` do projeto dbt e pode ser publicada com dbt docs. A mesma modelagem é executada em dois warehouses sem alteração de SQL — DuckDB (arquivo) e Postgres (cliente-servidor, em contêiner) —, trocando apenas o `target` do dbt. O Postgres é a fonte lida pelo Metabase, que não possui driver nativo para DuckDB, e essa dupla execução é a evidência prática de que a modelagem é portátil para o Snowflake.

6. Uso de recursos da AWS

Conforme o item 4.5, a solução utiliza o Amazon S3 para o armazenamento dos dados, organizados em três camadas (`raw`, `processed`, `curated`). A estrutura completa de nuvem foi descrita como código no template CloudFormation, que provisiona:

Recurso AWS	Papel na solução
S3 (3 buckets)	Data lake em camadas <code>raw/processed/curated</code> , com versionamento e criptografia
IAM Role	Permissões mínimas de acesso ao S3 e ao CloudWatch
Lambda	Função de ingestão (stub) que grava dados na camada <code>raw</code>
EventBridge	Agendamento diário da ingestão
CloudWatch Logs	Monitoramento e retenção de logs do pipeline

- três buckets S3, um por camada (`raw`, `processed` e `curated`), com criptografia SSE-S3 e bloqueio de acesso público;
- um papel IAM com permissão restrita às operações de leitura e escrita nesses buckets;
- um log group no Amazon CloudWatch para o monitoramento básico do pipeline;
- uma função AWS Lambda de ingestão, agendada por uma regra do Amazon EventBridge.

O pipeline inclui uma etapa de sincronização com o S3 (`src/cloud/s3_sync.py`, via `boto3`) que envia as camadas `raw`, `processed` e `curated` para o bucket quando as credenciais AWS e a variável `S3_BUCKET` estão definidas. Na execução local usamos o template enxuto `infra/cloudformation-s3-pipeline.yaml`, que cria apenas o bucket: a cada execução o pipeline provisiona a stack via CloudFormation e envia as três camadas para o S3 — na última execução, 11 arquivos.

Consideramos também a execução no AWS Academy (Learner Lab), ambiente usado no curso, que restringe a criação de papéis IAM (permite apenas o papel pré-existente `LabRole`), trava a região em `us-east-1` e não disponibiliza serviços gerenciados como o MWAA. Para esse cenário, disponibilizamos uma variante do template (`infra/cloudformation-academy.yaml`) que reutiliza o `LabRole` em vez de criar um papel, e utilizamos o `upload` para o S3, etapa que não exige papel algum (basta as credenciais temporárias do lab). Dessa forma, o requisito de usar um serviço da AWS é cumprido mesmo sob as restrições do Academy, enquanto a arquitetura completa (MWAA, SageMaker) permanece como proposta de referência no diagrama.

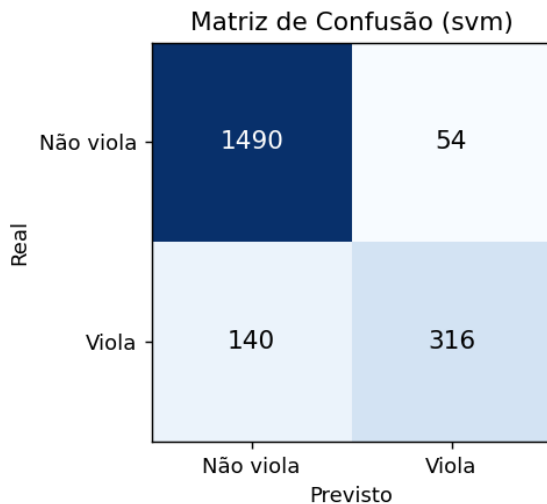
7. Aprendizagem de Máquina: modelos e métricas

A tarefa é de classificação binária. Seguindo o item 4.6, criamos baselines, implementamos o modelo na forma `hard-code` e depois com biblioteca, e comparamos os resultados.

Usamos os algoritmos vistos na disciplina: KNN, SVM e MLP. Implementamos o KNN do zero (apenas NumPy): ele guarda os exemplos e, para prever, calcula a distância euclidiana até todos os pontos de treino, seleciona os k vizinhos mais próximos (usamos $k = 15$) e decide pela votação da maioria. Em seguida rodamos o mesmo KNN com o scikit-learn, e as duas versões ficaram idênticas (concordância de 100% nas predições), o que nos dá confiança de que a implementação manual está correta. Como comparação, treinamos ainda um SVM (kernel RBF) e um MLP (rede neural). O SVM teve o melhor desempenho ($F1 = 0,77$; $ROC-AUC = 0,95$) e foi escolhido como modelo de produção.

Modelo	Acurácia	Precisão	Recall	F1	ROC-AUC
Baseline (classe majoritária)	0.772	0.0	0.0	0.0	0.5
Baseline (regra por prioridade)	0.771	0.4983	0.6294	0.5562	0.7211
KNN (hard-code)	0.838	0.7619	0.4211	0.5424	0.8821
KNN (scikit-learn)	0.838	0.7619	0.4211	0.5424	0.8821
SVM (produção)	0.903	0.8541	0.693	0.7651	0.9529
MLP	0.8875	0.7473	0.7654	0.7562	0.9443

As métricas foram escolhidas conforme a tarefa (classificação): acurácia, precisão, recall, F1 e ROC-AUC, além da matriz de confusão. Demos atenção especial ao recall, pois deixar de sinalizar um chamado que vai violar o SLA (falso negativo) é mais custoso para a operação do que um alarme falso.



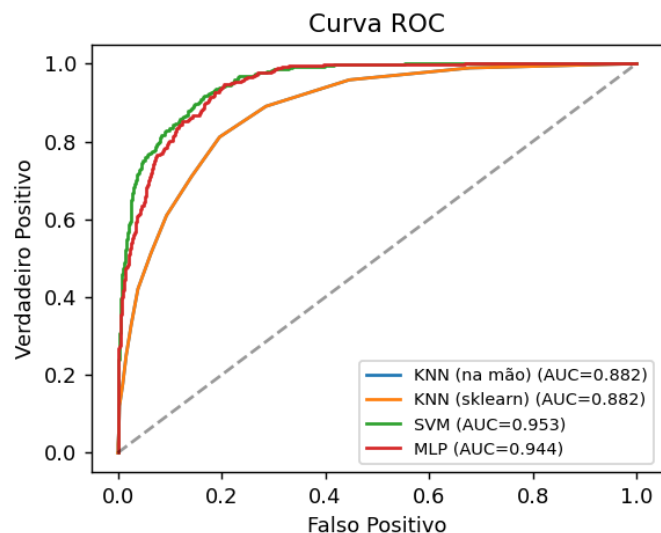


Figura 2. Matriz de confusão e curva ROC (conjunto de teste).

8. Dashboard e principais análises

O dashboard foi construído no Metabase sobre as tabelas do schema analytics. Ele reúne os indicadores principais (volume de chamados, taxa de violação de SLA, tempo médio de resolução e CSAT), a visualização dos dados tratados (violação por prioridade, categoria, canal e mês), os resultados do modelo (matriz de confusão e distribuição de risco) e uma fila priorizada pelos chamados de maior risco. O painel tem três filtros — prioridade, categoria e canal — ligados às colunas reais das tabelas (field filters), e não a texto livre. Eles recortam o painel inteiro: aplicando Prioridade = Crítica, por exemplo, a taxa de violação de SLA sai de 22,8% para 60,4%. O painel é provisionado por script, via API do Metabase, junto com o pipeline.

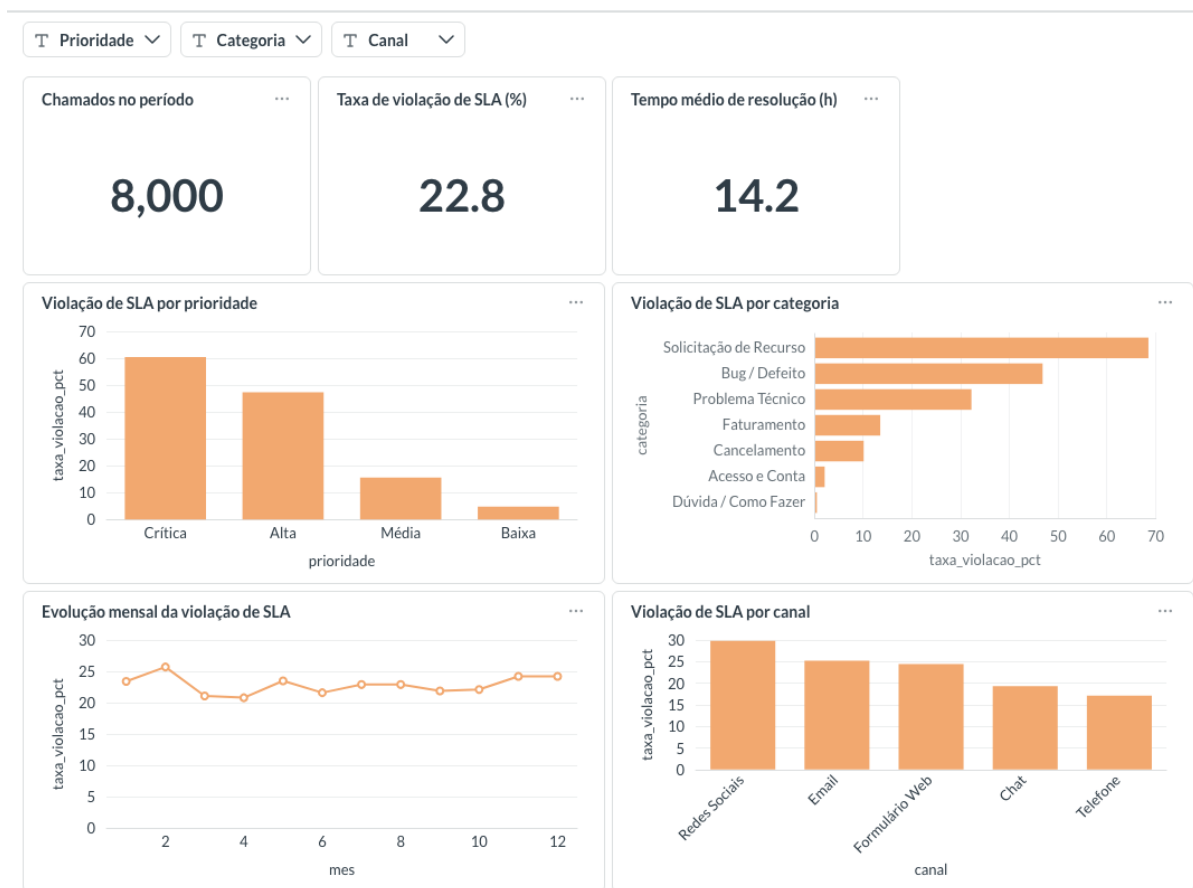


Figura 3a. Painel no Metabase: indicadores e violação de SLA por prioridade, categoria, canal e mês.

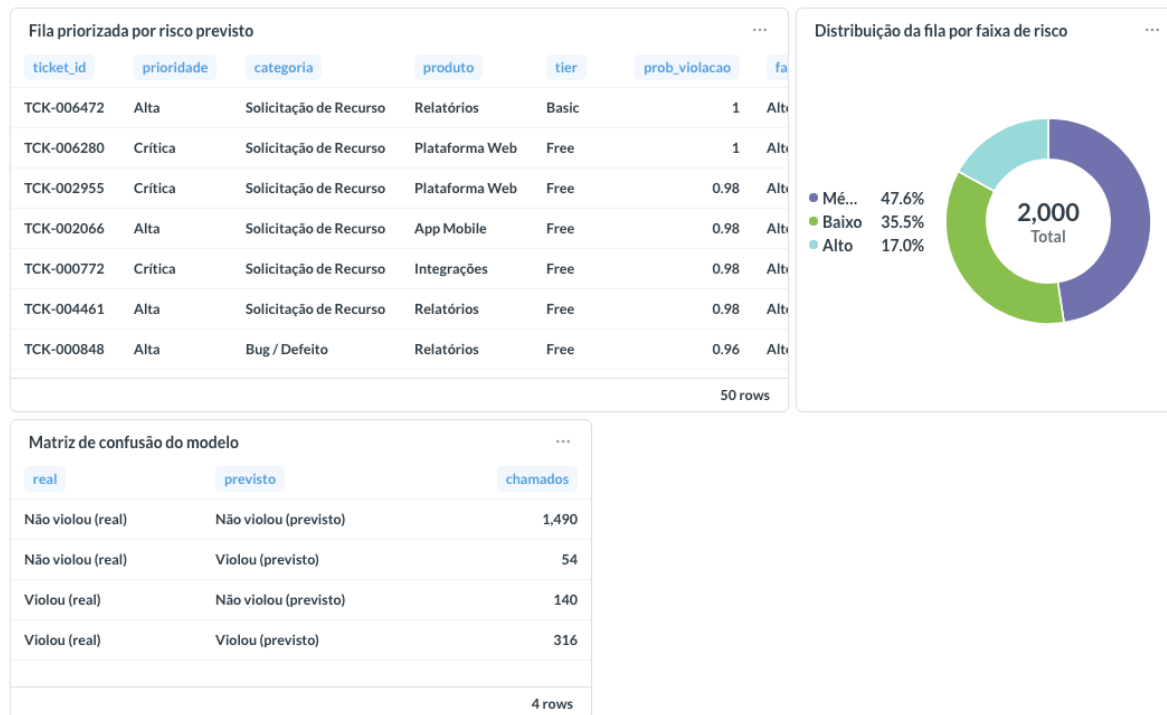


Figura 3b. Painel no Metabase: fila priorizada por risco previsto, distribuição da fila por faixa de risco e

matriz de confusão do modelo.

As análises mostram que as categorias Solicitação de Recurso e Bug/Defeito concentram as maiores taxas de violação, e que prioridade Crítica e Alta são as mais críticas. Na prática, o gestor usa a fila priorizada por risco para realocar agentes antes de o prazo estourar, e os gráficos por categoria/canal para decisões estruturais (reforço de equipe, base de conhecimento, automações).

9. Avaliação do pipeline e análise qualitativa

Além das métricas do modelo, avaliamos o pipeline como um todo: as etapas executam na ordem correta, os testes do dbt garantem a qualidade dos dados após a transformação, e existe rastreabilidade do dado bruto até a predição (raw -> processed -> marts -> predictions).

Análise qualitativa

- Acertos: chamados de prioridade Alta/Crítica em categorias demoradas (Bug/Defeito, Solicitação de Recurso) são corretamente sinalizados com alta probabilidade de violação.
- Erros: o modelo erra mais em casos de fronteira (prioridade Média com texto curto), gerando alguns falsos negativos.
- Possíveis causas: sobreposição natural entre as classes e ausência de atributos sobre a carga do agente específico e o histórico do cliente.
- Limitações dos dados: por serem sintéticos, não capturam toda a complexidade de textos reais (gírias, erros de digitação, múltiplos idiomas).
- Riscos de uso: o modelo pode reforçar vieses operacionais (priorizar sempre os mesmos perfis) e há risco de uso do score como única base de decisão.
- Melhorias futuras: incorporar TF-IDF/embeddings do texto, adicionar features de carga do time, calibrar o threshold por custo de negócio e monitorar drift do modelo.

10. Limitações e próximos passos

A camada de nuvem já está implementada e plugável: o S3 recebe as camadas de dados via boto3, a infraestrutura é descrita como código em CloudFormation, o dbt possui profiles para DuckDB, Postgres e Snowflake, e a orquestração roda em uma DAG de Airflow em contêiner, com execução registrada de nove tarefas em nove. Como próximos passos de escala, pretendemos: (1) executar a carga em um Snowflake real via o profile já preparado; (2) subir a DAG no Amazon MWAA; (3) treinar e servir o modelo no SageMaker; e (4) evoluir os atributos textuais com técnicas de PLN. Mesmo assim, a solução já cumpre o ciclo completo pedido no enunciado, da ingestão à visualização para a tomada de decisão.

Referências

- ENUNCIADO. Projeto Final Integrado: Pipeline de Dados em Nuvem para Aprendizagem de Máquina. IFG, Pós-Graduação em IA Aplicada, Módulo 2, 2026.
- dbt Labs. dbt documentation. Disponível em: <https://docs.getdbt.com>.
- Snowflake. Snowflake Documentation. Disponível em: <https://docs.snowflake.com>.
- Apache Airflow. Documentation. Disponível em: <https://airflow.apache.org/docs>.
- Amazon Web Services. AWS Documentation (S3, Lambda, CloudFormation, SageMaker, MWAA). Disponível em: <https://docs.aws.amazon.com>.
- PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. JMLR, v. 12, 2011.

- Metabase. Metabase Documentation. Disponível em: <https://www.metabase.com/docs>.